

Semantics and Equational Axiomatisation for Quantum Communication

Theo Wang*

theo.wang@spc.ox.ac.uk

University of Oxford

Oxford, United Kingdom

1 Overview of the Presentation

What. We study the semantics of quantum programs extended with classically controlled quantum communication: sending and receiving qubits, possibly depending on a classical measurement outcome. For example, the following is naturally expressed in imperative syntax, but not in the standard quantum circuit model.

```
p: qbit = in(); q: qbit = new(|0>);
if (measure(p) == |1>) { out(q); }
```

The goal is to develop an axiomatisation of program equivalence between communicating quantum programs.

Why. Quantum communication is an important programming primitive in distributed quantum computing [3]. Text-book quantum protocols such as teleportation [2] and BB84 [1] already implicitly use communication primitives. Prior work already treats quantum communication within quantum process calculi [4, 5, 8, 14], where communication is entangled with the additional complexity of concurrency.

How. We study the individual operations of quantum communication *in isolation*, as algebraic effects, following the tradition of Plotkin and Power [10]. We do so following the triangle summarised by fig. 1. We propose a new algebraic theory for quantum communication building on [12], and study an operational and a denotational model of the theory. Finally, we show that denotational and operational equalities coincide.

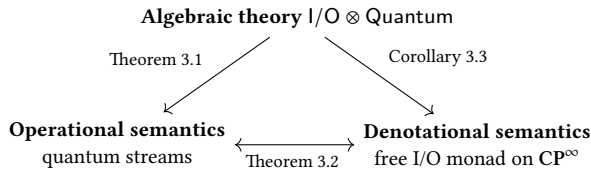


Figure 1. Three semantics of quantum communication and how they relate.

*This presentation is based on joint work with Sam Staton (Oxford) [13], presented for the first time at PPlanQC 2025.

2 Background: Axiomatising Quantum Computing

We already know how to build the same triangle for qubit quantum computing without communication. Our starting point is [12]’s axiomatisation for qubit quantum computing, Quantum, based on *parameterised* algebraic theories (PAT). PATs are a generic framework for axiomatising the behaviour of computations depending on certain resources in terms of operations and equations. Here, resources will correspond to qubits.

Operations $\text{op} : (p \mid m_1 \dots m_k)$ represent computations which take p qubits, classically choose one of the k branches, say, i , and return m_i qubits. Quantum has three operations: state preparation $\text{new} : (0 \mid 1)$, unitary evolution $\text{apply}_U : (n \mid n)$ for each $2^n \times 2^n$ unitary U , and classical control $\text{measure} : (1 \mid 0, 0)$.

Terms generated from the operations, follow the same intuition: a term $x_1 : m_1 \dots x_k : m_k \mid q_1 \dots q_p \vdash c$ takes p input qubits $q_1 \dots q_p$ and calls one of the k continuations $x_1 \dots x_k$ with the corresponding number of qubits. We can thus write for example the program

$$x : 1, y : 1 \mid q_1, q_2 \vdash \text{measure}(q_1, x(q_2), y(q_2))$$

which measures a q_1 and continues with x or y with q_2 based on the measurement outcome.

Equations are given in the form $\Gamma \mid \Delta \vdash c = c'$. Quantum for example specifies the effect of measuring a $|0\rangle$ qubit using the following equation:

$$x : 0, y : 0 \mid \cdot \vdash \text{new}(q.\text{measure}(q, x, y)) = x$$

Models: Denotational and Operational [12] proceeds to define a denotational model for the Quantum theory, where each term $x_1 : m_1 \dots x_k : m_k \mid q_1 \dots q_p \vdash c$ is interpreted as a complete positive unital (CPU) map

$$\llbracket c \rrbracket : \bigoplus_{i=1}^k \mathcal{M}_{2^{m_i}}(\mathbb{C}) \rightarrow \mathcal{M}_{2^p}(\mathbb{C}).$$

Remarkably, this model is *fully complete*: every CPU map of this type is the interpretation of some term c , and two CPU

maps are equal iff their term representation are derivably equal in the equational theory.

We can in fact give an equivalent operational model for Quantum. Briefly, configurations are of the form $\langle \rho, c \rangle$ where ρ is the program state, c the running program, and transitions $\rightarrow_p$ are probabilistic. We can then formulate in terms of $\rightarrow_p$ an operational notion of program equivalence $\approx_{qu}$ equivalent with denotational equality.

3 Semantics for Quantum Communication

We proceed to extend our language with communication primitives, and strive to recover the same triangle.

3.1 An Algebraic Theory for Quantum Communication

Extending Quantum with two operations: $\text{in} : (0 \mid 1)$ and $\text{out} : (1 \mid 0)$ lets us represent classically controlled communication. We can express the example program as $x : 0 \mid \vdash \text{in}(p.\text{new}(q.\text{measure}(p, x, \text{out}(q, x))))$. We further add equations to make in and out commute with every Quantum operation. What we now obtain is $\text{I/O} \otimes \text{Quantum}$, where $\otimes$ denotes the commutative combination of two theories. This choice is motivated by quantum theory. For instance,

$$\text{measure}(a, \text{out}(b, x), \text{out}(b, y)) = \text{out}(b, \text{measure}(a, x, y))$$

is required to derive the equivalence between: preparing a Bell pair, sending one qubit and discarding the other; and tossing a fair coin and sending the outcome as a qubit.

3.2 An Operational Model

The notion of communication that our theory soundly axiomatises can be demonstrated through an operational model. We work in CP , the category of finite dimensional C^* algebras and complete positive maps, constructed as in [11].

The operational semantics is based on a *quantum stream*, specifying the environment our program communicates with. The idea is then to have a global quantum state $\rho : \mathbb{C} \rightarrow S \otimes \text{qbit} \otimes H$, where S is the program state, H is the hidden stream state, and qbit is the next input qubit. Then, a stream is a state machine reacting to the program's communications: formally, a stream is specified by a hidden state type H_σ and transition maps $T_{\text{in}}(\sigma)$, $T_{\text{out}}(\sigma)$ for every communication trace $\sigma \in \{\text{in}, \text{out}\}^*$.

We can then build an operational semantics with configuration $\langle \rho, \sigma, c \rangle$, where ρ is the global state, $\sigma \in \{\text{in}, \text{out}\}^*$, c is the program. Given a stream specification t we can give a small step transition relation $\rightarrow_p^t$ defined inductively on the program. For example for the output operation, the rule is as in fig. 2: the output is performed, and the stream state is updated according to t . Finally, $\approx_{qu}$ extends naturally to communicating quantum programs, thus specifying a sound model of $\text{I/O} \otimes \text{Quantum}$.

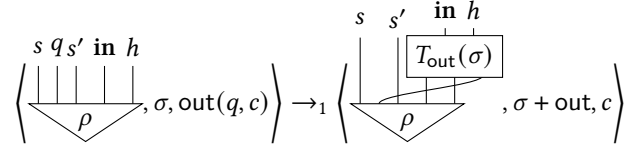


Figure 2. Operational Rule for out

Theorem 3.1. $\Gamma \mid \Delta \vdash c = c' \implies \Gamma \mid \Delta \vdash c \approx_{qu} c'$.

3.3 A Denotational Model

The Quantum I/O Monad In the tradition of algebraic effects [6, 10], we build an I/O monad $(T(X) = \mu Y. [\text{qbit}^{\text{in}}, Y] \oplus \text{qbit}^{\text{out}} \otimes Y \oplus X)$, where we annotate qbit for bookkeeping reasons) directly on top of CP. On the one hand, we can simplify the definition of T by unrolling the fixed point and by noting that $[\text{qbit}^{\text{in}}, Y] \cong \text{qbit}^{\text{in}} \otimes Y$ by compact closure and self-duality:

$$T(X) = \bigoplus_{\sigma \in \{\text{in}, \text{out}\}^*} (\text{qbit}^\sigma \otimes X)$$

where $\text{qbit}^{\text{in}, \text{out}, \dots} = \text{qbit}^{\text{in}} \otimes \text{qbit}^{\text{out}} \otimes \dots$. On the other hand, T is not well-defined because CP only has finite biproducts. To make it well-defined, we follow [9] and complete CP with infinite biproducts using a dcpo completion, obtaining CP^∞ . Now T is well-defined as a functor $\text{CP}^\infty \rightarrow \text{CP}^\infty$, which is in fact a monad.

Denotational Semantics We simply interpret each $\Gamma \mid \Delta \vdash c$ term as a morphism $\llbracket c \rrbracket : \llbracket \Delta \rrbracket \rightarrow T(\llbracket \Gamma \rrbracket)$ in the Kleisli category of T . The interesting case is the input operator $\llbracket \text{in} \rrbracket : \mathbb{C} \rightarrow T(\text{qbit})$, defined as

$$\llbracket \text{in} \rrbracket \triangleq \text{inj}^{\text{in}} \circ \left[\text{qbit}^{\text{in}} \cup \text{qbit} \right]$$

where the ‘cup’ is the unnormalised Bell state. Intuitively, $\llbracket \text{in} \rrbracket$ is supposed to take a new input; the cup bends the input wire up so it appears on the output side, as specified by the monad. It turns out this denotation corresponds exactly to the operational notion of equality, $\approx_{qu}$. We thus get:

Theorem 3.2. $\Gamma \mid \Delta \vdash c \approx_{qu} c' \iff \llbracket c \rrbracket = \llbracket c' \rrbracket$.

Corollary 3.3. $\Gamma \mid \Delta \vdash c = c' \implies \llbracket c \rrbracket = \llbracket c' \rrbracket$.

4 Limitations and Future Work

We have obtained a sound axiomatisation $\text{I/O} \otimes \text{Quantum}$ for one notion of quantum communication, presented in two equivalent ways: operationally and denotationally. However, unlike the quantum computing case, we do not have completeness – whether $\llbracket c \rrbracket = \llbracket c' \rrbracket \implies c = c'$ is open. Achieving this is our immediate next step. It would equally be interesting to relate this work to quantum process calculi, higher order quantum computing [7, 9], and their applications in verifying quantum protocols (e.g. [4]).

References

- [1] Charles H. Bennett and Gilles Brassard. 2014. Quantum cryptography: Public key distribution and coin tossing. *Theoretical Computer Science* 560 (2014), 7–11. <https://doi.org/10.1016/j.tcs.2014.05.025> Theoretical Aspects of Quantum Cryptography – celebrating 30 years of BB84.
- [2] Charles H. Bennett, Gilles Brassard, Claude Crépeau, Richard Jozsa, Asher Peres, and William K. Wootters. 1993. Teleporting an unknown quantum state via dual classical and Einstein-Podolsky-Rosen channels. *Phys. Rev. Lett.* 70 (Mar 1993), 1895–1899. Issue 13. <https://doi.org/10.1103/PhysRevLett.70.1895>
- [3] Marcello Caleffi, Michele Amoretti, Davide Ferrari, Jessica Illiano, Antonio Manzalini, and Angela Sara Cacciapuoti. 2024. Distributed quantum computing: A survey. *Computer Networks* 254 (Dec. 2024), 110672. <https://doi.org/10.1016/j.comnet.2024.110672>
- [4] Lorenzo Ceragioli, Fabio Gadducci, Giuseppe Lomurno, and Gabriele Tedeschi. 2024. Quantum Bisimilarity via Barbs and Contexts: Curbing the Power of Non-deterministic Observers. *Proc. ACM Program. Lang.* 8, POPL, Article 43 (jan 2024), 29 pages. <https://doi.org/10.1145/3632885>
- [5] Simon Gay and Rajagopal Nagarajan. 2004. Communicating Quantum Processes. arXiv:quant-ph/0409052 [quant-ph] <https://arxiv.org/abs/quant-ph/0409052>
- [6] Martin Hyland, Gordon Plotkin, and John Power. 2006. Combining effects: Sum and tensor. *Theoretical Computer Science* 357, 1 (2006), 70–99. <https://doi.org/10.1016/j.tcs.2006.03.013> Clifford Lectures and the Mathematical Foundations of Programming Semantics.
- [7] Aleks Kissinger and Sander Uijlen. 2019. A categorical semantics for causal structure. *Logical Methods in Computer Science* 15 (2019).
- [8] Marie Lalire and Philippe Jorrand. 2004. A Process Algebraic Approach to Concurrent and Distributed Quantum Computation: Operational Semantics. arXiv:quant-ph/0407005 [quant-ph] <https://arxiv.org/abs/quant-ph/0407005>
- [9] Michele Pagani, Peter Selinger, and Benoît Valiron. 2013. Applying quantitative semantics to higher-order quantum computing. *CoRR* abs/1311.2290 (2013). arXiv:1311.2290 <http://arxiv.org/abs/1311.2290>
- [10] Gordon Plotkin and John Power. 2002. Notions of Computation Determine Monads. In *Foundations of Software Science and Computation Structures*, Mogens Nielsen and Uffe Engberg (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 342–356.
- [11] Peter Selinger. 2007. Dagger Compact Closed Categories and Completely Positive Maps: (Extended Abstract). *Electronic Notes in Theoretical Computer Science* 170 (2007), 139–163. <https://doi.org/10.1016/j.entcs.2006.12.018> Proceedings of the 3rd International Workshop on Quantum Programming Languages (QPL 2005).
- [12] Sam Staton. 2015. Algebraic Effects, Linearity, and Quantum Programming Languages. In *Proceedings of the 42nd Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages* (Mumbai, India) (POPL '15). Association for Computing Machinery, New York, NY, USA, 395–406. <https://doi.org/10.1145/2676726.2676999>
- [13] Théo Wang. 2024. *Algebraic Theories and Quantum Communication*. Master’s thesis. University of Oxford, Oxford, United Kingdom. <https://theo.wang/works/2024-09-MSc-thesis>
- [14] Mingsheng Ying, Yuan Feng, Runyao Duan, and Zhengfeng Ji. 2010. An Algebra of Quantum Processes. arXiv:0707.0330 [quant-ph] <https://arxiv.org/abs/0707.0330>